

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Sistemas de Inteligencia Artificial · Trabajo Práctico 1

Métodos de Búsqueda

INTEGRANTES

Facundo Lasserre

Gianna Lucía Nacuzzi

Javier Bautista Peral Belmont

Manuel José Santamarina Balbín

Santiago Mesa Rubio

Contenido

- 01** 8-puzzle: estado y heurísticas
- 03** Diseño experimental
- 05** Demostración y elección del método

- 02** Sokoban: estado, métodos y heurísticas
- 04** Informada vs. no informada

LADO A · EJERCICIO 1

8-puzzle

Estructura de estado, heurísticas y métodos propuestos

Estructura del estado

Tupla inmutable de nueve posiciones, leída por filas.

El 0 representa el espacio vacío.

Dos estados son iguales si coinciden las nueve posiciones.

El nodo agrega padre, acción, profundidad y costo; esos datos no forman parte del estado.

Cada movimiento válido del vacío cuesta 1.

5	7	3
8	2	0
1	6	4



(5, 7, 3, 8, 2, 0, 1, 6, 4)

Cada ficha fuera de lugar exige al menos un movimiento

Heurística 1 · cantidad de fichas numeradas en una posición incorrecta

EJEMPLO

$$h_1(s) = 3$$



Tres fichas están fuera de su casilla objetivo.

1

Qué cuenta

Una unidad por cada ficha numerada mal ubicada.

2

Qué exige la solución

Cada ficha contada debe cambiar de posición al menos una vez.

3

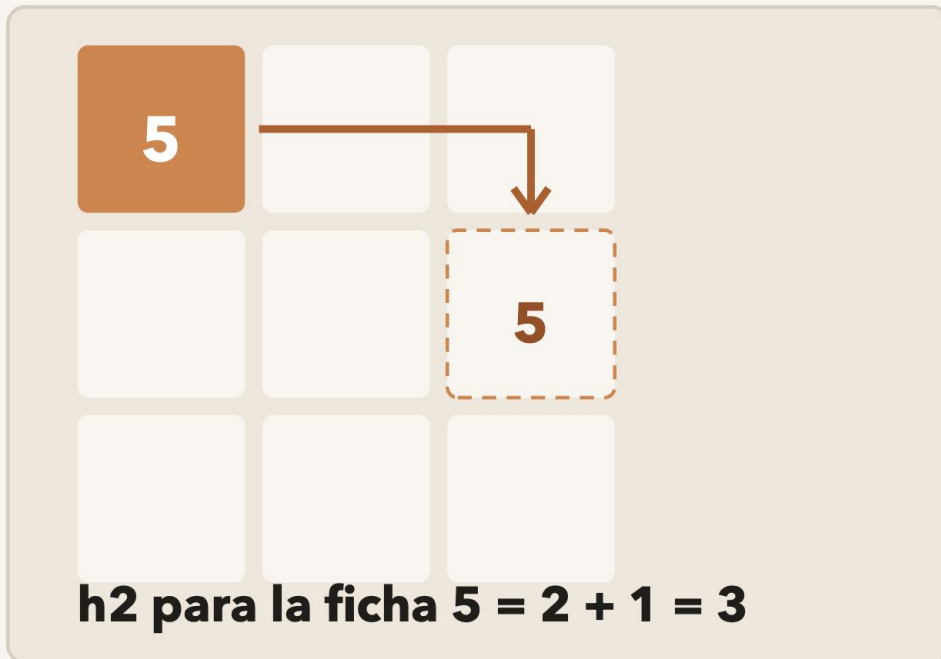
Conclusión

Un movimiento desplaza una sola ficha: el costo real nunca puede ser menor que h_1 .

Por eso es admisible: $h_1(s) \leq$ costo óptimo real. Si hay varios objetivos, se conserva la menor cota.

Manhattan cuenta los pasos ortogonales que todavía faltan

Heurística 2 · suma de distancias de cada ficha a su posición objetivo



1

Qué cuenta

Movimientos horizontales y verticales, ignorando bloqueos e interacciones.

2

Qué puede hacer una acción

Mover una sola ficha una casilla y reducir la suma, como máximo, en 1.

3

Conclusión

Para llevar la suma a cero hacen falta al menos h_2 movimientos.

Por eso es admisible: $h_2(s) \leq \text{costo óptimo real}$. Para los tres objetivos se conserva el menor valor.

Métodos propuestos para el 8-puzzle

MÉTODO

MOTIVO

BFS

Referencia completa y óptima con costo unitario

A* · fichas fuera de lugar

Primera búsqueda informada admisible

A* · Manhattan

Cota dominante, sin perder optimalidad

Greedy · Manhattan

Contrastar rapidez contra calidad de la solución

LADO B • EJERCICIO 2

Sokoban

Motor de búsqueda implementado y medido

Nivel, estado y nodo

COMPARTIDO

Nivel

Paredes, pisos y objetivos. Es inmutable: se calcula una sola vez y se comparte entre todos los nodos.

DINÁMICO

Estado

Posición de la persona más un frozen set de cajas: representación inmutable, hashable y comparable.

DE BÚSQUEDA

Nodo

Estado, padre, transición, profundidad y costo acumulado. Permite reconstruir el camino sin guardarlo.

Costo adoptado: cada movimiento válido de la persona cuesta 1, empuje incluido.

El tratamiento de repetidos depende del método

MÉTODO

PRIORIDAD

REPETIDOS

BFS

Menor profundidad

Marcar al insertar

DFS

Mayor profundidad

Marcar al insertar

Greedy

Menor h

Marcar al insertar

A*

Menor $g + h$

Conservar el mejor g y reabrir

Desempate determinista: UP · DOWN · LEFT · RIGHT

Las dos heurísticas admisibles

HEURÍSTICA 1 • MMM

Manhattan mínima caja–objetivo

$h(s)$ = mínima suma Manhattan entre asignaciones caja–objetivo

Conserva una asignación uno a uno. Ignora paredes, otras cajas y la posición del jugador.

Admisible y barata, pero optimista.

HEURÍSTICA 2 • SPA

Distancia al próximo empuje

$h(s)$ = camino real más corto hasta una postura de empuje válida

Contempla paredes y cajas actuales. Ignora el empuje y si acerca una caja al objetivo.

Admisible, débil y costosa de calcular.

Qué reporta el motor en cada corrida

Resultado de la búsqueda: éxito o fracaso.

Costo de la solución, movimientos y empujes.

Nodos expandidos, frontera final y frontera máxima.

Tiempo de procesamiento en segundos.

Camino completo y animación opcional.

```
jperalb ~/sia-tps/tp1 main ? → sia-tp1 --config
Status: success
Expanded nodes: 4
Frontier at end: 1
Maximum frontier: 2
Elapsed seconds: 0.000039
Solution cost: 3
Solution moves: 3
Solution pushes: 2
Solution:
1: RIGHT, pushed=false
2: RIGHT, pushed=true
3: RIGHT, pushed=true
Final state:
#####
#___@*#
#####
GIF saved to: output/solucion.gif (4 frames)
```

Configuración por archivo: nivel,
método y heurística

Seis configuraciones sobre tres niveles

CASOS

BFS, DFS, Greedy·MMM, Greedy·SPA, A*·MMM y A*·SPA.

PROTOCOLO

10 repeticiones por caso, cada una en un proceso nuevo y con el orden mezclado.

CORTE

1.000.000 de expansiones o 25 segundos.

REGISTRO

Resultado, costo, nodos, frontera máxima, camino y tiempo.



Base

Óptimo BFS: 43



Intermedio

Óptimo BFS: 78



Difícil

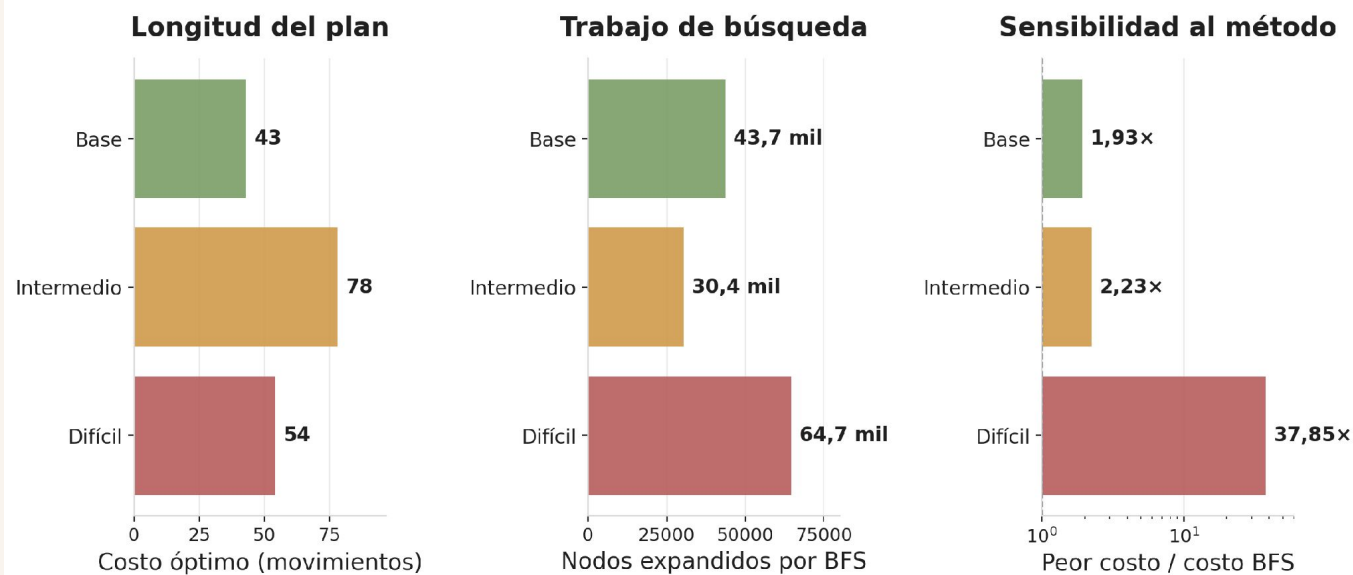
Óptimo BFS: 54

En Sokoban no hay una única dificultad

Longitud del plan, trabajo de búsqueda y sensibilidad al método ordenan los niveles de manera distinta

Los niveles son difíciles por razones distintas

La longitud del camino, el trabajo de búsqueda y la sensibilidad no crecen juntas



Base • level_03

19 pisos · 4 cajas · óptimo 43
BFS expande 43.709 · sensibilidad 1,93×

Intermedio • level_02

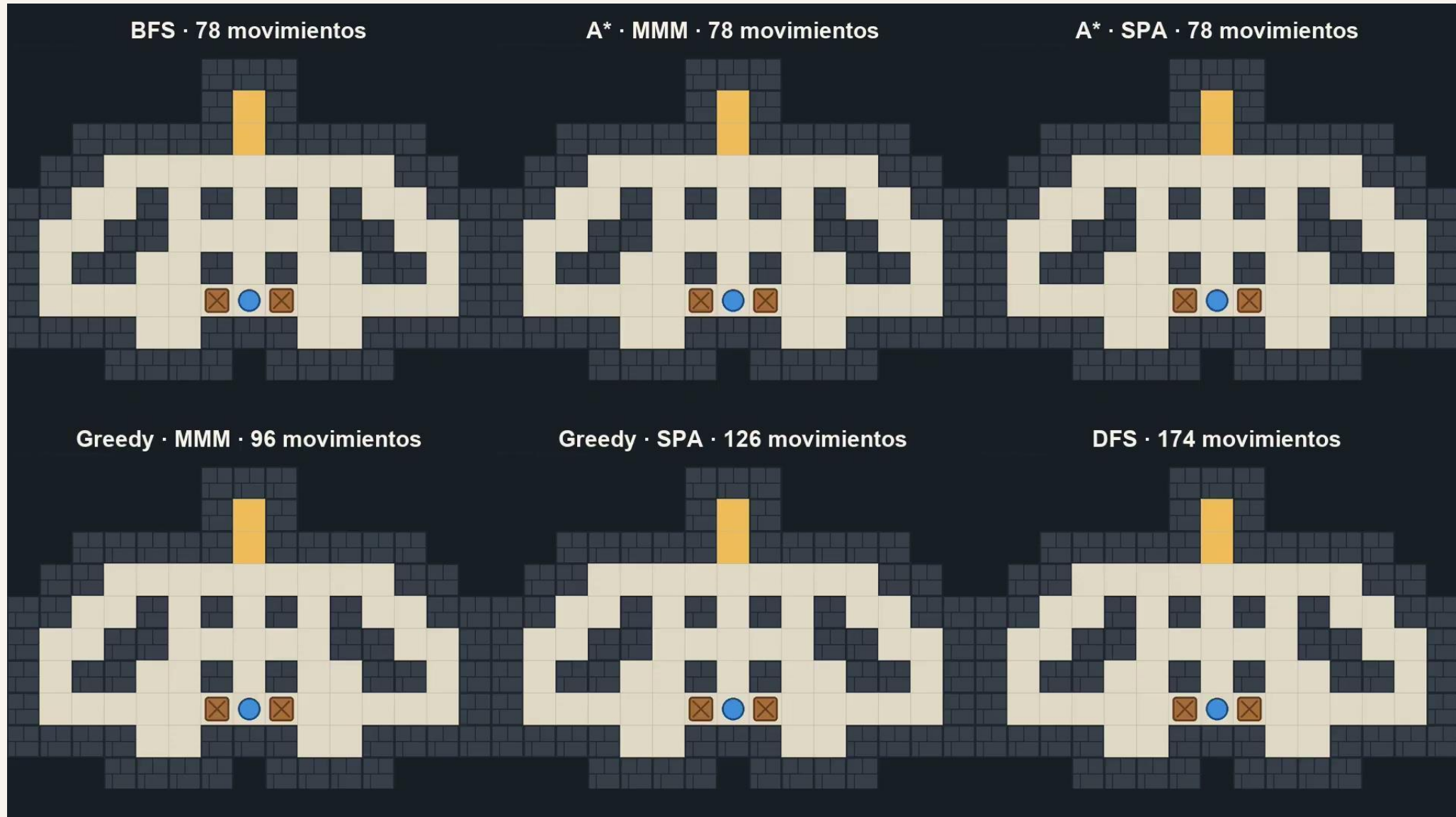
51 pisos · 2 cajas · óptimo 78
BFS expande 30.375 · sensibilidad 2,23×

Difícil • level_04

55 pisos · 2 cajas · óptimo 54
BFS expande 64.687 · sensibilidad 37,85×

El mismo nivel produce seis recorridos distintos

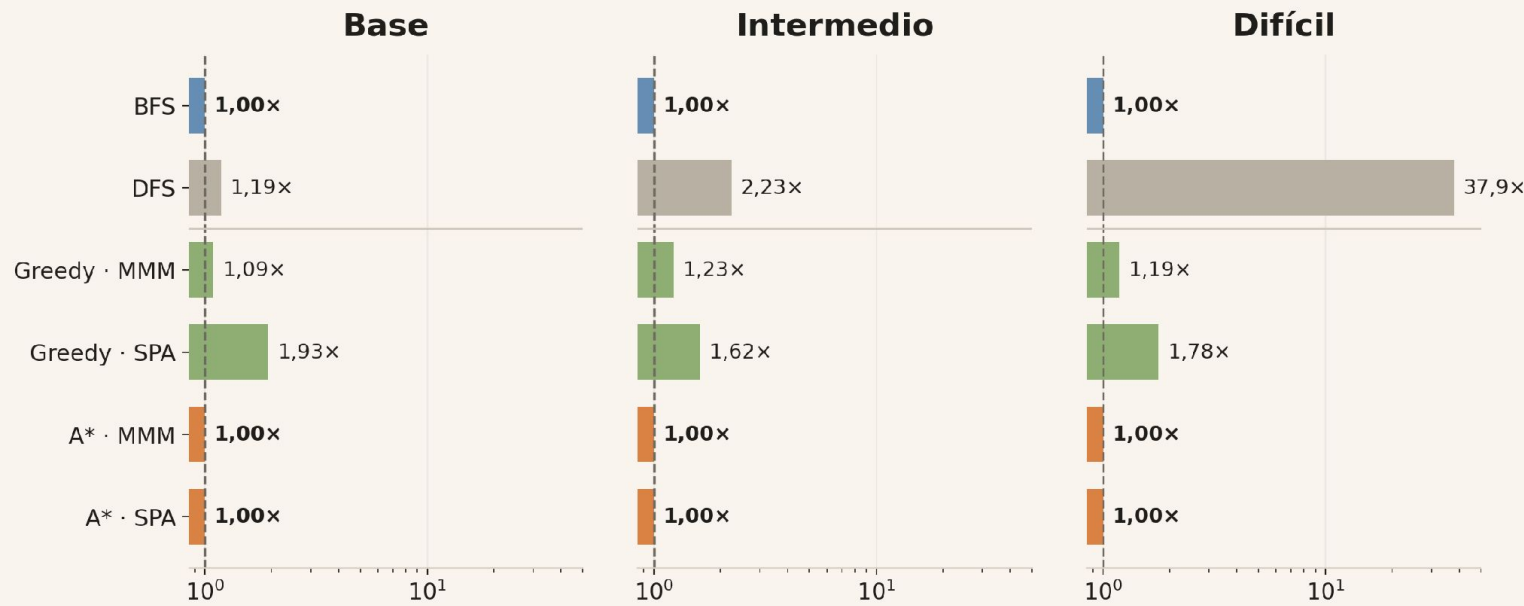
Nivel intermedio · seis métodos



Usar una heurística no garantiza una solución óptima

BFS y DFS: no informadas · Greedy y A*: informadas · costo relativo a BFS

Línea punteada = óptimo de BFS · separación horizontal = no informada / informada



Costo / costo BFS (escala logarítmica)

OPTIMALIDAD

BFS y A* coinciden en 43, 78 y 54 movimientos.

INTERCAMBIO

Greedy · MMM acepta entre 9 % y 23 % más movimientos.

INESTABILIDAD

DFS llega a 2.044 movimientos frente a un óptimo de 54.

Greedy prioriza h , no $g + h$: una heurística admisible no lo vuelve óptimo.

La calidad inestable de DFS se ve en el recorrido

Nivel difícil · DFS: 2.044 movimientos · A* + MMM: 54 movimientos



La información reduce trabajo, no siempre memoria

Valores relativos a BFS dentro de cada nivel · menos de 1 favorece al método

	Trabajo · nodos expandidos			Memoria aproximada · frontera máxima		
BFS	1,00×	1,00×	1,00×	1,00×	1,00×	1,00×
DFS	0,99×	0,069×	1,09×	0,036×	0,11×	0,37×
Greedy · MMM	0,099×	0,025×	0,013×	0,45×	0,089×	0,045×
Greedy · SPA	0,79×	0,74×	0,089×	3,16×	3,44×	0,96×
A* · MMM	0,98×	1,00×	0,71×	1,21×	1,17×	0,98×
A* · SPA	0,94×	0,88×	0,78×	1,40×	1,38×	1,44×
	Base	Intermedio	Difícil	Base	Intermedio	Difícil

Cada celda se divide por BFS en ese nivel · verde = menos · rojo = más

GREEDY · MMM

Reduce las expansiones entre 10× y 75× y mantiene fronteras menores.

A*

Conserva el costo óptimo, pero en base e intermedio expande casi como BFS.

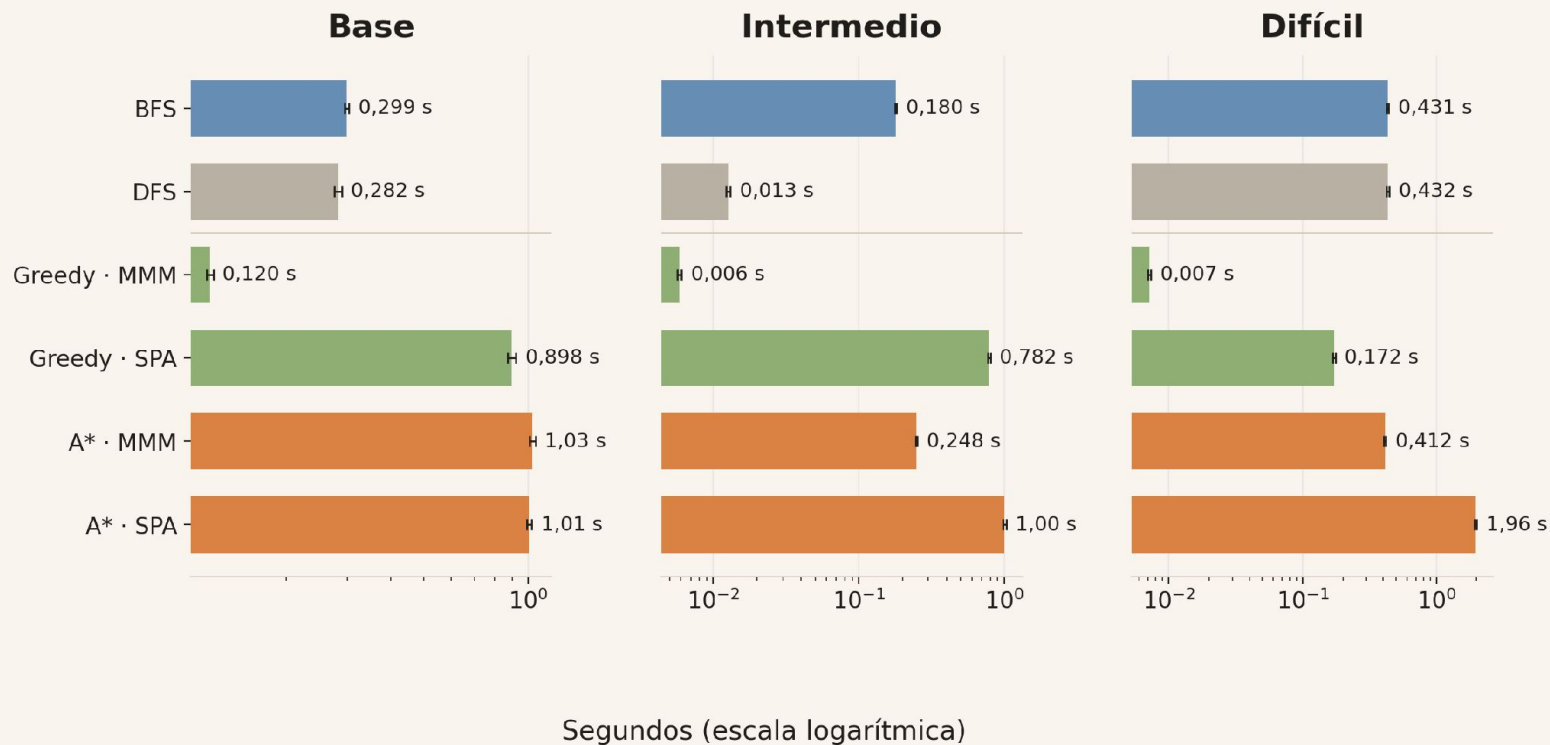
DFS

Guarda pocas alternativas, pero la calidad de su solución es inestable.

Menos expansiones no garantiza menos tiempo

Tiempo del algoritmo · 10 repeticiones por configuración · mismo equipo

Barra = promedio de 10 ejecuciones · error = una desviación estándar



MÁS RÁPIDO

Greedy · MMM obtiene el menor tiempo promedio en los tres niveles.

COSTO DE SPA

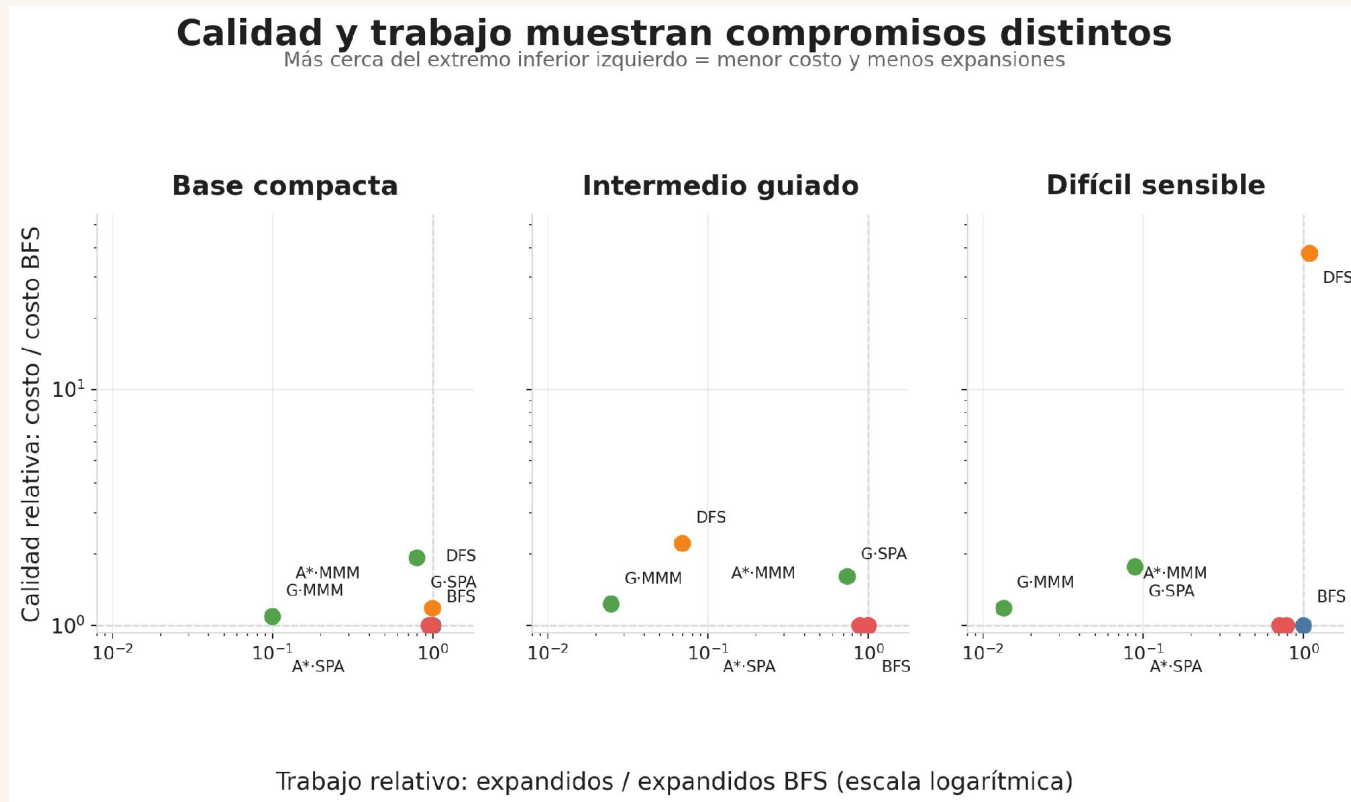
En A*, SPA tarda $\approx 4,0\times$ más en intermedio y $\approx 4,7\times$ en difícil.

RESULTADO

Las 180 de 180 ejecuciones terminaron con éxito.

Normalizar permite comparar decisiones

Calidad relativa = costo ÷ BFS · trabajo relativo = expandidos ÷ BFS · ejes logarítmicos



BFS queda en (1, 1): es la referencia.

A* conserva calidad relativa 1.

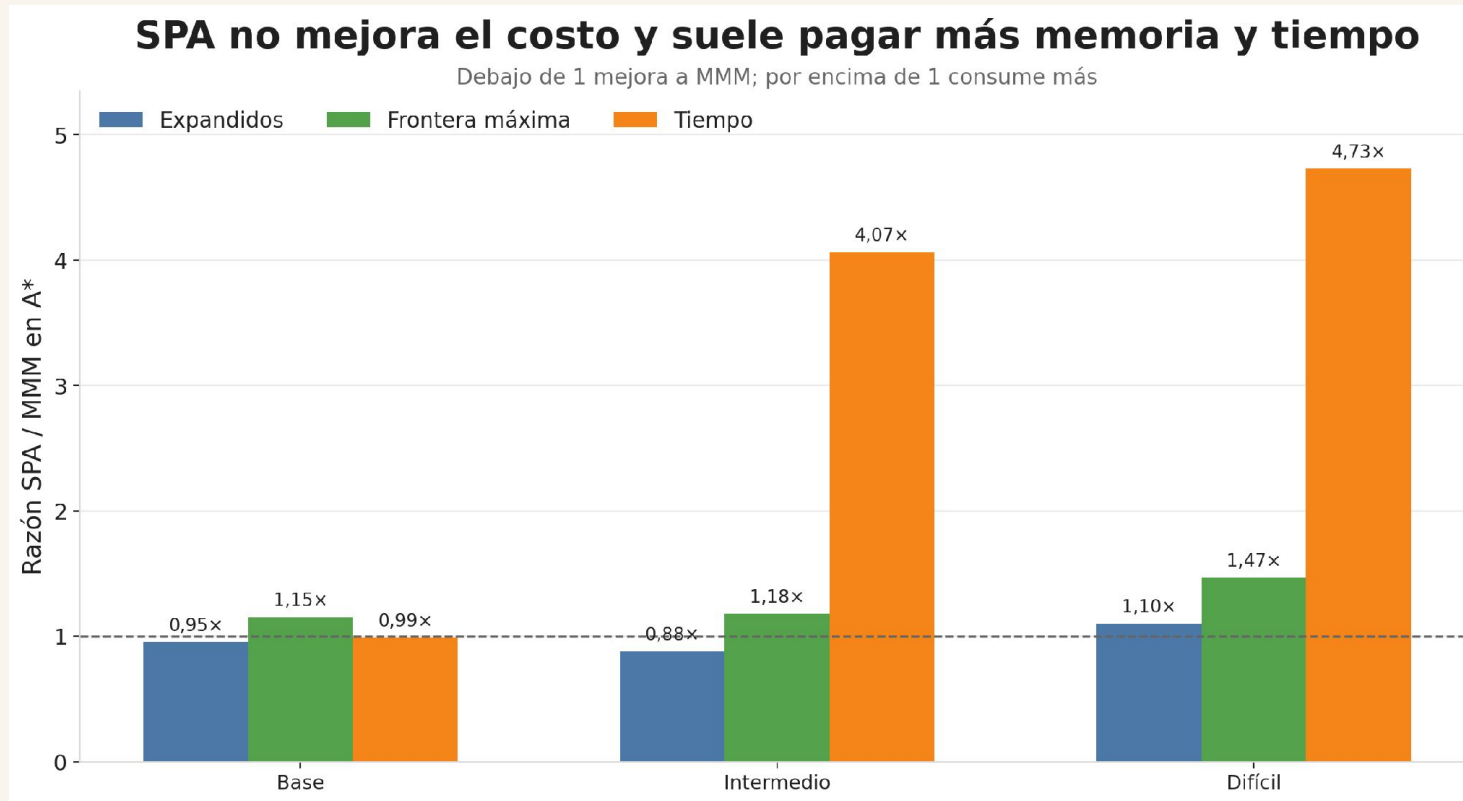
Greedy · MMM ahorra trabajo con poco sobrecosto.

DFS presenta calidad inestable sin ahorrar búsqueda.

El extremo inferior izquierdo combina menor costo y menos expansiones.

SPA frente a MMM dentro de A*

Razón SPA ÷ MMM · por debajo de 1 favorece a SPA; por encima de 1 consume más



MISMO COSTO

MMM y SPA obtienen 43, 78 y 54; por eso el costo no se grafica.

MENOS EXPANSIONES

SPA expande entre 5 % y 12 % menos en base e intermedio.

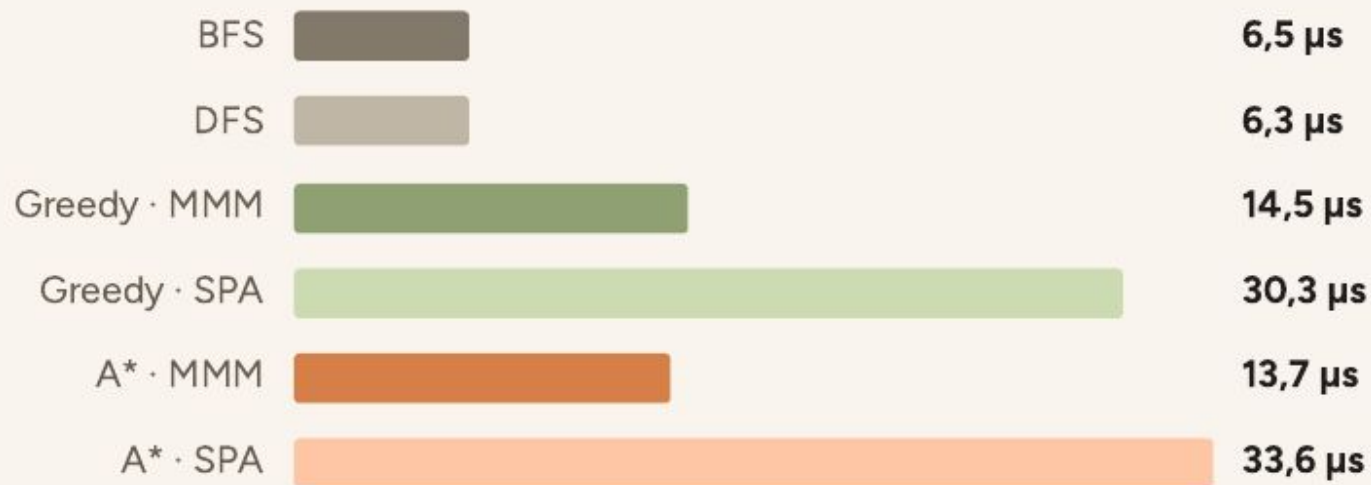
MAYOR COSTO TOTAL

Usa entre 15 % y 47 % más frontera y tarda hasta 4,73× más.

MMM ofrece el mejor balance general: mismo costo, menos memoria y sin BFS interna.

Dónde se paga cada heurística

Microsegundos por nodo expandido · promedio de los tres niveles



SIN HEURÍSTICA

Expandir un nodo cuesta alrededor de 6 μs: es el motor de búsqueda solo.

MMM

Duplica el costo por nodo con una asignación caja-objetivo.

SPA

Multiplica el costo por cinco: ejecuta una BFS interna en cada expansión.

A* · SPA es lento por lo que paga al evaluar h , no porque expanda más.

Calidad contra esfuerzo

Rangos observados entre los tres niveles

MÉTODO	SOBRECOSTO	NODOS FRENTE A BFS	FRONTERA MÁX. / EXPANDIDOS
BFS	0 %	1,0×	3 % a 7 %
DFS	+19 % a +3.685 %	1,09× a 14× menos	0,2 % a 5 %
Greedy · MMM	+9,3 % a +23,1 %	10× a 75× menos	10 % a 30 %
Greedy · SPA	+62 % a +93 %	1,3× a 11× menos	13 % a 53 %
A* · MMM	0 %	1,0× a 1,4× menos	3 % a 8 %
A* · SPA	0 %	1,1× a 1,3× menos	5 % a 10 %

A* reproduce el costo óptimo. Greedy · MMM acepta entre 9 % y 23 % de sobrecosto y expande hasta 75 veces menos nodos.

La elección depende del objetivo

1

BFS cuando el espacio es manejable y se busca optimalidad con bajo costo por nodo: fue más rápido que A* en los niveles base e intermedio, porque no paga el cálculo de una heurística.

2

A* • MMM cuando se busca optimalidad reduciendo el trabajo de búsqueda: reprodujo el costo de BFS y, en el nivel difícil, expandió aproximadamente un 29 % menos de nodos.

3

Greedy • MMM cuando importa reducir expansiones y se acepta una solución no óptima: expandió entre 10 y 75 veces menos nodos, con un sobre costo de entre 9 % y 23 %.

4

DFS cuando la prioridad es mantener una frontera pequeña: consume poca memoria aproximada, pero su calidad es inestable y puede devolver caminos extremadamente largos.

SPA no mejoró la calidad: su BFS interna redujo algunas expansiones, pero aumentó la frontera y el tiempo total.